

Rapport de testing

I. Recommandations

A. Tests à automatiser

Les deux tests que j'ai choisis d'automatiser sont la connexion et le panier.

- la connexion est le point essentiel qui réunit les parties de l'interface utilisateur et l'aspect sécurité du site. Elle est essentielle pour l'utilisateur car c'est la première interactions qu'il aura avec le site, pour l'entreprise elle est gage de sécurité et de gestion des utilisateurs. Elle est nécessaire pour ajouter les produits au panier et donc pour pouvoir passer commande.

- Le panier est la finalité du passage de l'utilisateur sur le site, l'étape qui va lui permettre de passer sa commande et à l'entreprise de pouvoir faire une vente. Sur les 3 points, ces deux-là sont les plus importants.

B. Préconisations pour la suite

- Je conseil pour la suite à minima de faire le dernier point (l'affichage du panier) en test manuel. Il reste quand même essentiel, moins que les deux cité précédemment mais je pense qu'il doit être vérifié, il fait partie d'une interaction importante avec l'utilisateur. Rendre les tests déjà automatisés plus complets en leur faisant vérifier l'aspect responsive, on ne peut pas négliger ce point là au vu du nombre d'utilisateur sur mobile et tablette.

- Je pense qu'il est nécessaire d'automatiser ce genre de test dit « essentiel », il permette de s'assurer rapidement du fonctionnement basique du site après chaque mise à jour ou nouvelle fonctionnalité, l'ajout d'un nouveau produit pourrait venir perturber tout sa, et d'un point de vu économique c'est un investissement sur le long terme, si le budget le permet je conseil donc d'automatiser au moins les points que j'ai cité précédemment.

II. Bilan de campagne de validation : tests automatisés

Document revu par :

Nom	Fonction	Version	Signature
Jordan	QA Engineer	1.0.0	

Contexte :

La campagne effectuée ici est pour assurer la réussite du site de vente en ligne d'Éco Bliss Bath. C'est un site d'e-commerce spécialisé dans la vente de produits de beauté éco responsables dont le produit principal est un savon solide. Il y a une part de test manuel et une autre d'automatique, les tests ont été fait sur Google chrome / windows 11.

Objectif(s) :

L'objectif ici est de rendre ces tests plus rapides à réaliser en ciblant des fonctions essentielles du site. L'automatisation permettra de vérifier que le site fonctionne toujours de manière optimale après des modifications comme un nouveau produit ou des modifications sur ceux existant (hausse de prix, changement de description).

Test effectués

- Smoke test :

- _ *Charger la page principale avec succès :*
 - Vérifie que la page principale contient le logo
- _ *La présence des boutons :*

- Visite la page principale
- Vérifie la visibilité des boutons :
 - du logo,
 - avis,
 - connexion,
 - inscription,
 - accueil.

_ Le contenu de la page des produits :

- Clique sur le bouton produits de la page d'accueil
- Vérifie que l'url comporte « /products »
- Vérifie la visibilité des infos :
 - nom du produits
 - description
 - l'image
 - le prix
- le lien vers la page du produit

_ Le contenu d'une page produit :

- Visite la page des produits,
- clic sur le premier produit de la liste,
- vérifie la présence des éléments :
 - l'image, le nom, la description, les

arômes, ingrédients, le prix, que le stock soit positif, sur quel produit, et le bouton d'ajout au panier.

- Le panier :

_ Ajout d'un produit en rupture de stock :

- Se connecter
- Visiter la page du produit N°3
- Clique sur le bouton d'ajout au panier
- Visiter la page panier
- Vérifie que l'article dans le panier contient le nom du produit N°3

_ Ajout d'un produit au panier sans connexion :

- Visiter la page du produit N°6
- Clique sur le bouton d'ajout au produit
- Vérifie que l'on est redirigé vers la page de

connexion

_ Ajout d'un produit au panier :

- Se connecter
- Visite la page du produit N°7
- Clique sur le bouton d'ajout au panier
- Visite la page panier
- Vérifie que l'article dans le panier contient le nom du produit N°7

_ Ajout d'un produit avec un nombre supérieur au stock :

- Se connecter
- Visite le page du produit N°7
- Change la quantité pour 3
- Clique sur le bouton d'ajout au panier
- Visite la page panier
- Vérifie que l'article dans la panier contient le nom

du produit N°7

_ Suppression d'un produit dans la panier :

- Se connecter
- Visite la page du produit N°7
- Clique sur le bouton d'ajout au panier
- Visite la page panier
- Clique sur la corbeille

_ Modification de la quantité dans la panier :

- Se connecter
- Visite le page du produit N°7
- Change la quantité pour 3
- Clique sur le bouton d'ajout au panier
- Visite la page panier
- Efface la valeur
- Vérifie que la quantité c'est bien remis à 1

automatiquement

_ Validation de commande :

- Se connecter
- Visite la page du produit N°8
- Clique sur la bouton d'ajout au panier
- Visite la page panier

- Rempli le formulaire de validation de commande avec des données aléatoires grâce à "faker"
- Vérifie la redirection vers la page de confirmation

- Connexion :

_ Connexion basique :

- Visite la page login
- Rempli le mail et le mot de passe
- Clique sur le bouton de connexion
- Vérifie la redirection et que le token existe

_ Connexion avec une erreur mail :

- Visite la page login
- Rempli le mail avec une erreur
- Rempli le bon mot de passe
- Clique sur le bouton de connexion
- Vérifie que l'API renvoie bien un message d'erreur

```

7  -click
    (xhr) POST 401 http://localhost:8081/login
8  wait 1000
9  wait @loginRequest
10 -assert expected 401 to equal 401
11 -assert expected Invalid credentials. to equal Invalid credentials.

```

_ Connexion avec une erreur mail :

- Visite la page login
- Rempli le bon mail
- Rempli le mauvais mot de passe
- Clique sur le bouton de connexion
- Vérifie que l'API renvoie bien un message d'erreur

_ Essai d'inscription :

- Visite la page d'enregistrement
- Rempli tous les champs avec des données aléatoires grâce à "faker"
- Clique sur le bouton de validation
- Vérifie la redirection et que le logo de déconnexion soit visible

_ *Laisser un avis :*

- Se connecter
- Visite la page avis
- Sélectionne la 4ème étoile
- Rempli le titre et le commentaire
- Clique sur le bouton de validation
- Vérifie que l'avis apparaît sur la page

- **API :**

_ *Connexion basique :*

- Visite la page login
- Rempli le mail et le mot de passe
- Clique sur le bouton de connexion
- Vérifie la redirection et que le token existe
- Vérifie la réponse de l'api " *POST, /login* "

```
7 -click
  (xhr) ● POST 200 http://localhost:8081/login loginRequest
8 wait @loginRequest
  (page load) --page loaded--
  (new url) http://localhost:8080/
  (fetch) ● GET 200 https://ra-f.fontawesome.com/releases/v5.15.4/css/free.min.css?token=bb652ddaa6
  (new url) http://localhost:8080/#/
  (xhr) ● GET http://localhost:8081/products/random
9 -assert expected 200 to equal 200
10 url
11 -assert expected http://localhost:8080/#/ to include /
12 window
13 its .localStorage.authToken
14 -assert expected fake-token-test to exist
```

_ *Récupère le panier :*

- Se connecter
- Visite la page produit N°8
- Ajoute le produit au panier
- Visite la page panier
- Récupère la réponse de l'api " *GET, /orders* "

```

11 -assert expected <a> to be visible
    (page load) --page loaded--
    (new url) http://localhost:8080/
    (fetch) ● GET 200 https://na-f.fontawesome.com/releases/v5.15.4/css/free.min.css?token=bb652dda6
    (new url) http://localhost:8080/#/
    (xhr) ● GET 200 http://localhost:8081/products/random
12 visit /products/8
13 get [data-cy="detail-product-add"]
14 -click
15 visit /cart
    (xhr) ● GET 200 http://localhost:8081/products/8
16 wait 1000
    (xhr) ● GET 200 http://localhost:8081/products/random
    (xhr) ● GET 200 http://localhost:8081/me
    (xhr) ● GET 200 http://localhost:8081/orders
17 wait @getCart
    ♀ -assert expected 200 to equal 200

```

_ Validation de commande :

- Se connecter
- Ajout d'un produit au panier
- Visite la page panier
- Rempli les champs d'adresse, code postale et ville
- Appuie sur le bouton de validation
- Récupère la réponse de l'api " *POST, /orders* "

```

21 -click
    (xhr) ● POST 200 http://localhost:8081/orders
22 wait @createOrder
    (new url) http://localhost:8080/#/confirmation
23 -assert expected 200 to equal 200
24 url
25 -assert expected http://localhost:8080/#/confirmation to include /confirmation

```

_ Ajout d'un produit au panier :

- Se connecter
- Visite la page produit N°8
- Ajoute le produit au panier

– *Suppression d'un produit dans le panier :*

- Se connecter
- Visite la page N°7
- Ajoute le produit au panier
- Visite la page panier
- Clique sur la corbeille
- Vérifie la réponse de l'api " *DELETE*,

/orders/{id}/delete "

```
17  get  [data-cy="cart-line"]
18  find  [data-cy="cart-line-delete"]
19  -click

(xhr)  ● DELETE 200 http://localhost:8081/orders/56/delete Deleted
20  wait  @Deleted
21  -assert  expected 200 to equal 200
```

– *Modification de la quantité :*

- Se connecter
- visite la page produit N°7
- Change la quantité pour 3 exemplaires
- Clique sur le bouton d'ajout
- Visite la page panier
- Efface la quantité
- Vérifie que la valeur est passé à 1
- Vérifie la réponse de l'api " *PUT*,

orders/{id}/change-quantity "

```
18  visit  /cart
19  wait  1000

(xhr)  ● GET 200 http://localhost:8081/me
(xhr)  ● GET 200 http://localhost:8081/orders
20  get  [data-cy="cart-line"]
21  find  [data-cy="cart-line-quantity"]
22  -clear
23  get  [data-cy="cart-line"]
24  find  [data-cy="cart-line-quantity"]
25  -assert  expected <input.ng-untouched.ng-
valid.ng-dirty> to have value '1'
26  wait  @quantity

(xhr)  ● PUT 200 quantity
http://localhost:8081/orders/53/change-quantity
27  -assert  expected 200 to equal 200
```


_ Récupère la liste des produits :

- Visite la page produit
- Vérifie la réponse de l'api " GET, /products "

```
✓ page produits
> ROUTES (1)
> BEFORE EACH
Open in IDE
▼ TEST BODY
1  visit /products
2  wait 1000
   (xhr) ● GET 200 http://localhost:8081/products
3  wait @getProducts
4  -assert expected 200 to equal 200
5  -assert expected [ Array(8) ] to be an array
6  get [data-cy="product-link"]
7  eq 0
8  -click
   (new url) http://localhost:8080/#/products/3
   (xhr) ● GET 200 http://localhost:8081/products/3
   (xhr) ● GET 200 http://localhost:8081/products/random
9  url
```

_ Récupère la page d'un produit :

- Se connecter
- Visite la page du produit N°7
- Vérifie la réponse de l'api " GET, /products/7 "

```
11 -assert expected <a> to be visible
    (page load) --page loaded--
    (new url) http://localhost:8080/
    (fetch) ● GET 200 https://ka-f.fontawesome.com/releases/v5.15.4/css/free.min.css?token=bb652dda6
    (new url) http://localhost:8080/#/
    (xhr) ● GET 200 http://localhost:8081/products/random
12 visit /products/7
13 wait @getCart
    (xhr) ● GET 200 http://localhost:8081/products/7
    (xhr) ● GET http://localhost:8081/products/random
14 -assert expected 200 to equal 200
```

_ *Laissez un avis :*

- Se connecter
- Visite la page avis
- Rempli le formulaire d'avis
- Clique sur le bouton d'envoi
- Vérifie la réponse de l'api " *POST, /reviews* "

```
(xhr) ● POST 200 http://localhost:8081/reviews Postreviews
23 wait 1000
(xhr) ● GET 200 http://localhost:8081/reviews
24 wait @Postreviews
25 -assert expected 200 to equal 200
26 -contains super produit!
27 -assert expected <h2> to be visible
28 -contains Que des produits de qualité, je recommande!
29 -assert expected <p> to be visible
```

_ *Récupère la liste des avis :*

- Se connecter
- Visite la page d'avis
- Vérifie la réponse de l'api " *GET, /reviews* "

```
12 visit /reviews
13 wait 1000
(xhr) ● GET 200 http://localhost:8081/reviews Getreviews
14 wait @Getreviews
15 -assert expected 200 to equal 200
```

– Vérifie l'inscription :

- Visite la page d'inscription
- Rempli les champs avec " Faker "
- Clique sur le bouton de validation
- Vérifie la réponse de l'api " *POST, /register* "

```
13 -click
(xhr) ● POST 200 http://localhost:8081/register registerRequest
14 wait 1000
15 wait @registerRequest
16 -assert expected 200 to equal 200
17 url
18 -assert expected http://localhost:8080/#/register to include /
19 get [data-cy="nav-link-logout"]
20 -assert expected <a> to be visible
(xhr) ● POST 200 http://localhost:8081/login
(page load) --page loaded--
(new url) http://localhost:8080/
(fetch) ● GET 200 https://ra-f.fontawesome.com/releases/v5.15.4/css/free.min.css?token=bb652ddaa6
(new url) http://localhost:8080/#/
```

– Test de faille XSS :

- Visite la page inscription
- Rempli les champs avec " <script> alert

("XSS")</script>

- Vérifie que le body ne contient pas de <script>

```
12 get [data-cy="register-submit"]
13 -click
14 wait 1000
(xhr) ● POST 200 http://localhost:8081/register
(xhr) ● POST http://localhost:8081/login
15 get body
16 -assert expected <body> not to contain <script>
```

Renouvelle l'opération avec la page login, d'inscription et avis.

Résultats de tests

Le résultat des tests montre plusieurs problèmes à régler.

- Stocks

On peut ajouter un produit même s'il est en rupture. Le test du panier “_ Ajout d'un produit en rupture de stock” devrait retourner une erreur.

L'affichage du stock peut aller en négatif au lieu d'afficher un “rupture de stock”

Il y a également une **erreur critique** dans la partie avis, il y a une faille XSS dans le formulaire permettant de poster un avis. Il faudra la corriger en priorité pour des questions évidentes de sécurité.

```
20 -type <script>alert("XSS")</script>
21 get [data-cy="review-submit"]
22 -click
23 wait 1000
   (xhr) POST 200 http://localhost:8081/reviews
   (xhr) GET 200 http://localhost:8081/reviews
24 get body
25 -assert expected <body> not to contain <script>

! AssertionError

Timed out retrying after 4000ms: expected '<body>' not to contain '<script>'

cypress/e2e/connexion.cy.js:185:24

183 |     cy.getBySel('review-submit').click();
184 |     cy.wait(1000)
> 185 |     cy.get('body').should('not.contain', '<script>');
      |                               ^
186 |     cy.on('window:alert', (txt) => {
187 |         expect(txt).to.not.include('<script>');
188 |     });

> View stack trace
Print to console
```

Anomalie :

On peut ajouter un produit au panier en choisissant 0 quantité, ce qui fait un produit dans le panier sans quantité et à 0 euros.

Confiance

Le projet en l'état actuel est un NO GO au vue de la criticité des anomalies, Il faut commencer par résoudre le problème de la faille Xss qui est pour moi la plus importante et critique.

Il y a également un problème sur la partie responsive, les menus n'apparaissent pas sur la page principal, comme citer plus haut dans le rapport, au vue du nombre de clients potentiel sur ces supports, il est essentiel qu'il soit optimal et intuitif.